

JavaScript Design Patterns — মডার্ন আর্কিটেকচার, ক্লিন কোড ও স্কেলেবিলিটি

সফটওয়্যার ডেভেলপমেন্টে রিযুজ্যাবল, টেস্টেবল ও মেইনটেইনেবল কোড লেখার জন্য ডিজাইন প্যাটার্ন অপরিহার্য। এটি কোনো কপি-পেস্ট কোড নয়, বরং বারবার ঘটা সফটওয়্যার সমস্যার একটি প্রমাণিত স্ট্রাকচারাল সমাধান। এই চ্যাপ্টারে আমরা **Creational (Factory, Singleton, Builder)**, **Structural (Adapter, Decorator, Facade, Proxy)**, **Behavioral (Observer, Pub/Sub, Strategy, Command, State, Chain)**, Architectural (MVC, Repository, Service Layer, Dependency Injection) এবং React ও Node.js-এর বাস্তব প্যাটার্নসমূহ সম্পূর্ণ কোড সহ শিখব।

★ 1. Creational Patterns

★ 2. Structural Patterns

★ 3. Behavioral Patterns

★ 4. Module & Singleton

★ 5. Factory & Builder

★ 6. Observer & Pub/Sub

★ 7. Strategy & Command

★ 8. Dependency Injection

★ 9. Repository & Service

★ 10. React & Enterprise

Part 01 — Design Patterns Fundamentals & Categories (37.1 – 37.8)

37.1 Design Pattern কী?

ধরো তুমি একটি বড় application বানাচ্ছে। তোমার code-এ আছে:

★ APPLICATION DOMAIN ENTITIES

User
Product
Cart
Payment
Database
API
Authentication
Notification

সবকিছু যদি একসাথে লিখো:

JAVASCRIPT Source

JAVASCRIPT

// 5000+ Lines

তাহলে project maintain করা কঠিন হয়ে যাবে। Design Pattern সাহায্য করে code-কে:

★ SOFTWARE QUALITY GOALS & CODE HEALTH

Organized
Reusable
Maintainable
Scalable
Testable

রাখতে।

37.2 Real-Life Example

ধরো একটি বাড়ি বানাবে। Architect প্রতিবার নতুন করে ভাববে না:

💡 “দরজা কোথায় রাখব? সিঁড়ি কীভাবে বানাব?”

বরং established architectural principles ব্যবহার করবে। Software Design Pattern-ও অনেকটা একই।

🌟 PROBLEM-TO-SOLUTION PATTERN MAPPING

```
Problem
↓
Known Pattern
↓
Structured Solution
```

37.3 Design Pattern মানে Ready-Made Code নয়

এটি খুব গুরুত্বপূর্ণ। Design Pattern:

🌟 SOFTWARE ARCHITECTURE DIAGRAM

❌ Copy-paste code

নয়। বরং:

🌟 SOFTWARE QUALITY GOALS & CODE HEALTH

- ✅ Design idea
- ✅ Structure
- ✅ Communication pattern
- ✅ Reusable approach

একই pattern বিভিন্ন project-এ আলাদাভাবে implement করা যায়।

37.4 Design Patterns-এর প্রধান Categories

সাধারণভাবে GoF (Gang of Four) অনুযায়ী:

🌟 SOFTWARE ARCHITECTURE DIAGRAM

```
Design Patterns
├── Creational
├── Structural
└── Behavioral
```

37.5 Creational Patterns

Object কীভাবে তৈরি হবে সেটি নিয়ে কাজ করে। Common:

🌟 SOFTWARE ARCHITECTURE DIAGRAM

```
Factory
Abstract Factory
Builder
Singleton
Prototype
```

37.6 Structural Patterns

Objects/classes কীভাবে একে অপরের সাথে structure তৈরি করবে। Common:

✦ SOFTWARE ARCHITECTURE DIAGRAM

Adapter
Decorator
Facade
Proxy
Composite
Bridge
Flyweight

37.7 Behavioral Patterns

Objects কীভাবে communicate করবে এবং behavior manage করবে। Common:

✦ SOFTWARE ARCHITECTURE DIAGRAM

Observer
Strategy
Command
State
Iterator
Mediator
Chain of Responsibility
Template Method

37.8 JavaScript-এ সবচেয়ে গুরুত্বপূর্ণ Patterns

তোমার Full Stack Development-এর জন্য আমরা বিশেষভাবে দেখব:

✦ SOFTWARE ARCHITECTURE DIAGRAM

🔥 Module
🔥 Singleton
🔥 Factory
🔥 Builder
🔥 Observer
🔥 Pub/Sub
🔥 Strategy
🔥 Adapter
🔥 Decorator
🔥 Facade
🔥 Proxy
🔥 Command
🔥 Chain of Responsibility
🔥 State
🔥 Dependency Injection

37.9 Module Pattern

Module Pattern-এর মূল উদ্দেশ্য:

💡 Related code এক জায়গায় রাখা এবং প্রয়োজনীয় অংশই বাইরে expose করা।

আগে JavaScript-এ:

JAVASCRIPT Source

JAVASCRIPT

```
const userModule = (() => {

  let users = [];

  function addUser(user) {
    users.push(user);
  }

  function getUsers() {
    return users;
  }

  return {
    addUser,
    getUsers
  };

})();
```

এখানে:

★ SOFTWARE ARCHITECTURE DIAGRAM

users

private। কিন্তু:

★ SOFTWARE ARCHITECTURE DIAGRAM

addUser()
getUsers()

public।

37.10 Module Pattern কেন দরকার?

ধরো:

★ SOFTWARE ARCHITECTURE DIAGRAM

users
products
payments
cart

সব global scope-এ রাখলে conflict হতে পারে। Module:

★ APPLICATION DOMAIN ENTITIES

User Module
Product Module
Payment Module
Cart Module

এভাবে code organize করতে সাহায্য করে।

37.11 Modern ES Module

Modern JavaScript-এ আলাদা Module Pattern লেখার প্রয়োজন অনেক কমেছে।

♦ user.js

JAVASCRIPT Source

JAVASCRIPT

```
export function createUser(name) {  
  return {  
    name  
  };  
}
```

♦ app.js

JAVASCRIPT Source

JAVASCRIPT

```
import { createUser } from "./user.js";  
  
const user = createUser("Ripon");  
  
console.log(user);
```

এটিই modern modular JavaScript!

37.12 Singleton Pattern

Singleton-এর উদ্দেশ্য:

💡 একটি class/object-এর একটিসমাত্র shared instance রাখা।

Real-life:

✦ SOFTWARE ARCHITECTURE DIAGRAM

```
Application  
  ↓  
Configuration  
  ↓  
One shared instance
```

✦ Simple Example

JAVASCRIPT Source

JAVASCRIPT

```
class Database {  
  
  static instance;  
  
  constructor() {  
  
    if (Database.instance) {  
      return Database.instance;  
    }  
  
    Database.instance = this;  
  }  
  
}
```

Use:

JAVASCRIPT Source

JAVASCRIPT

```
const db1 = new Database();  
const db2 = new Database();  
  
console.log(db1 === db2);
```

Output:

✦ SOFTWARE ARCHITECTURE DIAGRAM

```
true
```

37.13 Singleton কোথায় ব্যবহার হতে পারে?

যেমন:

★ SOFTWARE ARCHITECTURE DIAGRAM

Configuration
Logger
Cache
Database connection manager
Application state

কিন্তু সবকিছুর জন্য Singleton ব্যবহার করা উচিত নয়। কারণ excessive global shared state testing এবং maintainability কঠিন করতে পারে।

37.14 Factory Pattern

Factory Pattern object creation-এর logic আলাদা করে। ধরো:

JAVASCRIPT Source

JAVASCRIPT

```
function createUser(type) {  
  
  if (type === "admin") {  
    return {  
      role: "admin",  
      permissions: ["read", "write", "delete"]  
    };  
  }  
  
  if (type === "user") {  
    return {  
      role: "user",  
      permissions: ["read"]  
    };  
  }  
  
}
```

Use:

JAVASCRIPT Source

JAVASCRIPT

```
const admin = createUser("admin");  
const user = createUser("user");
```

37.15 Factory-এর Real-Life Example

ধরো payment system:

★ SOFTWARE ARCHITECTURE DIAGRAM

```
createPayment("card")
createPayment("paypal")
createPayment("bank")
```

Factory:

JAVASCRIPT Source

JAVASCRIPT

```
function createPayment(type) {

  switch (type) {

    case "card":
      return new CardPayment();

    case "paypal":
      return new PayPalPayment();

    case "bank":
      return new BankPayment();

    default:
      throw new Error("Unknown payment type");

  }
}
```

Client-কে জানতে হচ্ছে না object কীভাবে তৈরি হচ্ছে।

37.16 Builder Pattern

যখন object অনেক configuration/optional property নিয়ে তৈরি হয়, তখন Builder usefull ধরো:

JAVASCRIPT Source

JAVASCRIPT

```
const user = {
  name: "Ripon",
  age: 25,
  email: "ripon@example.com",
  role: "admin",
  active: true
};
```

অনেক field থাকলে Builder:

JAVASCRIPT Source

JAVASCRIPT

```
class UserBuilder {

  constructor() {
    this.user = {};
  }

  setName(name) {
    this.user.name = name;
    return this;
  }

  setEmail(email) {
    this.user.email = email;
    return this;
  }

  setRole(role) {
    this.user.role = role;
    return this;
  }

  build() {
    return this.user;
  }
}
```

Use:

JAVASCRIPT Source

JAVASCRIPT

```
const user = new UserBuilder()
  .setName("Ripon")
  .setEmail("ripon@example.com")
  .setRole("admin")
  .build();
```

37.17 Builder-এর গুরুত্বপূর্ণ Feature

এখানে:

JAVASCRIPT Source

JAVASCRIPT

```
return this;
```

দেওয়ার কারণে chaining করা যায়।

★ SOFTWARE ARCHITECTURE DIAGRAM

```
setName()  
↓  
setEmail()  
↓  
setRole()  
↓  
build()
```

এটাকে **method chaining** বলা হয়।

37.18 Prototype Pattern

JavaScript নিজেই prototype-based language। Prototype Pattern-এর idea হলো existing object থেকে নতুন object তৈরি করা।

JAVASCRIPT Source

JAVASCRIPT

```
const userPrototype = {  
  greet() {  
    console.log(`Hello ${this.name}`);  
  }  
};  
  
const user1 = Object.create(userPrototype);  
  
user1.name = "Ripon";  
  
user1.greet();
```

Output:

★ SOFTWARE ARCHITECTURE DIAGRAM

```
Hello Ripon
```

এখানে user1 prototype থেকে behavior পেয়েছে।

37.19 Adapter Pattern

Adapter incompatible interface-কে compatible interface-এ convert করে। Real-life:

 SOFTWARE ARCHITECTURE DIAGRAM

```

graph TD
    A[USB-C Laptop] --> B[Adapter]
    B --> C[HDMI Monitor]
  
```

Software-এও একই ধারণা। ধরো:

JAVASCRIPT Source

JAVASCRIPT

```

class OldPaymentSystem {
  makePayment(amount) {
    return `Paid ${amount}`;
  }
}

```

কিন্তু নতুন system চায়:

JAVASCRIPT Source

JAVASCRIPT

```

pay(amount)

```

Adapter:

JAVASCRIPT Source

JAVASCRIPT

```

class PaymentAdapter {
  constructor(oldSystem) {
    this.oldSystem = oldSystem;
  }

  pay(amount) {
    return this.oldSystem.makePayment(amount);
  }
}

```

Use:

JAVASCRIPT Source

JAVASCRIPT

```

const oldSystem = new OldPaymentSystem();

const payment = new PaymentAdapter(oldSystem);

console.log(payment.pay(1000));

```

37.20 Adapter কোথায় Useful?

 SOFTWARE ARCHITECTURE DIAGRAM

```

graph TD
    A[Old API] --> B[Adapter]
    B --> C[New Application]
  
```

Useful যখন:

- Legacy code আছে
- Third-party API-এর interface আলাদা
- Library replace করতে চাও
- Multiple data source unify করতে চাও

37.21 Decorator Pattern

Decorator existing object-এর behavior modify/extend করে। ধরো:

JAVASCRIPT Source

JAVASCRIPT

```
function basicUser() {  
  return {  
    name: "Ripon"  
  };  
}
```

Decorator:

JAVASCRIPT Source

JAVASCRIPT

```
function addAdminPermission(user) {  
  
  return {  
    ...user,  
    role: "admin"  
  };  
}
```

Use:

JAVASCRIPT Source

JAVASCRIPT

```
const user = basicUser();  
  
const admin = addAdminPermission(user);  
  
console.log(admin);
```

37.22 Real-Life Decorator

ধরো coffee:

★ SOFTWARE ARCHITECTURE DIAGRAM

```
Coffee  
↓  
+ Milk  
↓  
+ Sugar  
↓  
+ Cream
```

Software:

★ SOFTWARE ARCHITECTURE DIAGRAM

```
Base Function  
↓  
Decorator 1  
↓  
Decorator 2  
↓  
Decorator 3
```

একটি existing behavior-এর ওপর নতুন behavior layer করা হয়।

37.23 Higher-Order Function as Decorator

JavaScript-এ function দিয়েও decorator pattern করা যায়।

JAVASCRIPT Source

JAVASCRIPT

```
function withLogging(fn) {  
  
    return function (...args) {  
  
        console.log("Function called");  
  
        return fn(...args);  
    };  
  
}
```

Original:

JAVASCRIPT Source

JAVASCRIPT

```
function add(a, b) {  
    return a + b;  
}
```

Decorate:

JAVASCRIPT Source

JAVASCRIPT

```
const loggedAdd = withLogging(add);  
  
loggedAdd(2, 3);
```

Output:

🌟 SOFTWARE ARCHITECTURE DIAGRAM

Function called

Return:

🌟 SOFTWARE ARCHITECTURE DIAGRAM

5

37.24 Facade Pattern

Facade complex system-এর সামনে simple interface দেয়। ধরো online order system:

★ SOFTWARE ARCHITECTURE DIAGRAM

Order

- Inventory
- Payment
- Shipping
- Notification
- Database

Client-কে সব system আলাদাভাবে call করতে না দিয়ে:

JAVASCRIPT Source

JAVASCRIPT

```
class OrderFacade {  
  
  constructor() {  
    this.inventory = new Inventory();  
    this.payment = new Payment();  
    this.shipping = new Shipping();  
  }  
  
  placeOrder(order) {  
  
    this.inventory.reserve(order);  
    this.payment.charge(order);  
    this.shipping.ship(order);  
  
  }  
}
```

Client:

JAVASCRIPT Source

JAVASCRIPT

```
const orderSystem = new OrderFacade();  
  
orderSystem.placeOrder(order);
```

37.25 Facade-এর মূল ধারণা

★ SOFTWARE ARCHITECTURE DIAGRAM

Complex System



Facade



Simple API

37.26 Proxy Pattern

JavaScript-এ Proxy built-in feature। Proxy দিয়ে object-এর operation intercept করা যায়।

JAVASCRIPT Source

JAVASCRIPT

```
const user = {
  name: "Ripon"
};

const proxy = new Proxy(user, {

  get(target, property) {

    console.log(`Reading: ${property}`);

    return target[property];
  }

});
```

Use:

JAVASCRIPT Source

JAVASCRIPT

```
console.log(proxy.name);
```

Output conceptually:

★ SOFTWARE ARCHITECTURE DIAGRAM

```
Reading: name
Ripon
```

37.27 Proxy-এর Real Use Cases

★ SOFTWARE ARCHITECTURE DIAGRAM

```
Validation
Logging
Access control
Reactive systems
Caching
Data transformation
```

Vue-এর reactive system-এর মতো frameworks-এ Proxy-এর concepts গুরুত্বপূর্ণ।

Part 04 — Behavioral Patterns: Observer, Pub/Sub, Strategy, Command, State & Chain (37.28 – 37.41)

37.28 Observer Pattern

Observer Pattern-এর মূল idea:

💡 একটি object-এর state পরিবর্তন হলে interested observers-কে notify করা।

Real-life:

★ SOFTWARE ARCHITECTURE DIAGRAM

```
YouTube Channel
  ↓
New Video
  ↓
Subscribers
  ↓
Notification
```

37.29 Observer Example

JAVASCRIPT Source

JAVASCRIPT

```
class Subject {  
  
  constructor() {  
    this.observers = [];  
  }  
  
  subscribe(observer) {  
    this.observers.push(observer);  
  }  
  
  notify(data) {  
  
    this.observers.forEach(observer => {  
      observer(data);  
    });  
  
  }  
  
}
```

Use:

JAVASCRIPT Source

JAVASCRIPT

```
const news = new Subject();  
  
news.subscribe(data => {  
  console.log("User 1:", data);  
});  
  
news.subscribe(data => {  
  console.log("User 2:", data);  
});  
  
news.notify("New article published");
```

Output:

★ SOFTWARE ARCHITECTURE DIAGRAM

```
User 1: New article published  
User 2: New article published
```

37.30 Observer কোথায় ব্যবহার হয়?

★ SOFTWARE ARCHITECTURE DIAGRAM

```
DOM Events  
State Management  
UI Updates  
Notifications  
WebSocket Events  
Event Systems
```

React-এর state-driven UI এবং event-driven systems বুঝতে Observer concept helpfull

37.31 Pub/Sub Pattern

Observer-এর সাথে related হলেও Pub/Sub-এ publisher এবং subscriber সরাসরি একে অপরকে জানে না।

★ SOFTWARE ARCHITECTURE DIAGRAM

```
Publisher  
↓  
Event Bus  
↓  
Subscriber
```

37.32 Simple Pub/Sub

JAVASCRIPT Source

JAVASCRIPT

```
class EventBus {

  constructor() {
    this.events = {};
  }

  subscribe(event, callback) {

    if (!this.events[event]) {
      this.events[event] = [];
    }

    this.events[event].push(callback);
  }

  publish(event, data) {

    const callbacks = this.events[event] || [];

    callbacks.forEach(callback => {
      callback(data);
    });

  }
}
```

Use:

JAVASCRIPT Source

JAVASCRIPT

```
const bus = new EventBus();

bus.subscribe("login", user => {
  console.log("Welcome", user.name);
});

bus.publish("login", {
  name: "Ripon"
});
```

Output:

🌟 SOFTWARE ARCHITECTURE DIAGRAM

Welcome Ripon

37.33 Observer vs Pub/Sub

Observer	Pub/Sub
Subject knows observers	Publisher usually doesn't know subscribers
Direct relationship	Event channel/message bus
Simpler	More decoupled
State changes useful	Event communication useful

37.34 Strategy Pattern

Strategy Pattern:

💡 একই কাজ করার multiple algorithm/strategy থাকলে runtime-এ প্রয়োজন অনুযায়ী strategy নির্বাচন করা।

ধরো payment:

🌟 SOFTWARE ARCHITECTURE DIAGRAM

Card
PayPal
Bank
Crypto

একটি interface/contract-এর মাধ্যমে:

JAVASCRIPT Source

JAVASCRIPT

```
class PaymentProcessor {  
  
  constructor(strategy) {  
    this.strategy = strategy;  
  }  
  
  pay(amount) {  
    return this.strategy.pay(amount);  
  }  
  
}
```

Strategies:

JAVASCRIPT Source

JAVASCRIPT

```
const cardPayment = {  
  pay(amount) {  
    return `Card payment: ${amount}`;  
  }  
};  
  
const bankPayment = {  
  pay(amount) {  
    return `Bank payment: ${amount}`;  
  }  
};
```

Use:

JAVASCRIPT Source

JAVASCRIPT

```
const processor =  
  new PaymentProcessor(cardPayment);  
  
console.log(processor.pay(1000));
```

37.35 Strategy-এর Real-Life Example

Delivery system:

🌟 SOFTWARE ARCHITECTURE DIAGRAM

Standard Delivery
Express Delivery
Same-Day Delivery

Customer অনুযায়ী strategy change:

JAVASCRIPT Source

JAVASCRIPT

```
const delivery =  
  new DeliveryService(expressDelivery);
```

মূল business code বদলাতে হচ্ছে না।

37.36 Command Pattern

Command Pattern:

💡 কোনো operation-কে object হিসেবে represent করা।

ধরো text editor:

✦ SOFTWARE ARCHITECTURE DIAGRAM

Type
Copy
Paste
Undo
Redo

Command:

JAVASCRIPT Source

JAVASCRIPT

```
class AddTextCommand {  
  
  constructor(editor, text) {  
    this.editor = editor;  
    this.text = text;  
  }  
  
  execute() {  
    this.editor.add(this.text);  
  }  
  
  undo() {  
    this.editor.remove(this.text);  
  }  
  
}
```

এখানে action-টি object হয়ে গেল।

37.37 Command কোথায় Useful?

✦ SOFTWARE ARCHITECTURE DIAGRAM

Undo/Redo
Task Queue
Transactions
Menus
Buttons
Job Processing

37.38 State Pattern

একটি object-এর behavior তার current state-এর ওপর নির্ভর করলে State Pattern usefull Real-life:

★ SOFTWARE ARCHITECTURE DIAGRAM

Order

- Pending
- Paid
- Shipped
- Delivered

State পরিবর্তনের সাথে behavior পরিবর্তিত হয়।

JAVASCRIPT Source

JAVASCRIPT

```
class Order {  
  
  constructor() {  
    this.state = "pending";  
  }  
  
  pay() {  
  
    if (this.state === "pending") {  
      this.state = "paid";  
    }  
  
  }  
  
}
```

37.39 Chain of Responsibility

একটি request একাধিক handler-এর chain-এর মধ্য দিয়ে যায়।

★ SOFTWARE ARCHITECTURE DIAGRAM

```
Request  
↓  
Handler 1  
↓  
Handler 2  
↓  
Handler 3  
↓  
Response
```

37.40 Real-Life Example: Authentication

★ TIERED ENTERPRISE ARCHITECTURE FLOW

```
Request  
↓  
Check Token  
↓  
Check Permission  
↓  
Check Role  
↓  
Controller
```

Node/Express middleware system এই ধারণার সাথে খুব closely related। Example:

JAVASCRIPT Source

JAVASCRIPT

```
app.use(authMiddleware);  
app.use(roleMiddleware);  
app.get("/admin", controller);
```

37.41 Middleware as Chain

Express-এর conceptually:

SOFTWARE ARCHITECTURE DIAGRAM

```
Request
↓
Middleware 1
↓
Middleware 2
↓
Middleware 3
↓
Route Handler
```

প্রতিটি middleware:

JAVASCRIPT Source

JAVASCRIPT

```
function middleware(req, res, next) {

  // work

  next();
}
```

এটি Chain of Responsibility concept বোঝার জন্য খুব usefull

Part 05 — Architectural Patterns & Clean Software Engineering (37.42 – 37.55)

37.42 Dependency Injection

Dependency Injection বা DI হলো:

💡 কোনো object-এর প্রয়োজনীয় dependency নিজে তৈরি না করে বাইরে থেকে provide করা।

খারাপ coupling:

JAVASCRIPT Source

JAVASCRIPT

```
class UserService {

  constructor() {
    this.database = new MySQLDatabase();
  }

}
```

এখানে UserService নিজেই database তৈরি করছে।

37.43 Dependency Injection Example

Better:

JAVASCRIPT Source

JAVASCRIPT

```
class UserService {

  constructor(database) {
    this.database = database;
  }

}
```

তরপর:

JAVASCRIPT Source

JAVASCRIPT

```
const database = new MySQLDatabase();

const userService =
  new UserService(database);
```

37.44 DI কেন ভালো?

Testing সহজ:

JAVASCRIPT Source

JAVASCRIPT

```
const fakeDatabase = {
  getUser() {
    return {
      id: 1,
      name: "Test User"
    };
  }
};
```

তারপর:

JAVASCRIPT Source

JAVASCRIPT

```
const service =
  new UserService(fakeDatabase);
```

Real database দরকার নেই। এখানে Testing + Design Pattern একসাথে কাজ করছে।

37.45 Dependency Inversion-এর সাথে Relation

Dependency Injection হলো implementation technique। Dependency Inversion Principle হলো architectural principle। সহজভাবে:

🌟 SOFTWARE ARCHITECTURE DIAGRAM

High-level code



Abstraction



Low-level implementation

এতে tight coupling কমে।

37.46 Repository Pattern

Full Stack/Backend development-এ খুব useful। Database logic আলাদা layer-এ রাখা:

✦ TIERED ENTERPRISE ARCHITECTURE FLOW

```
Controller
↓
Service
↓
Repository
↓
Database
```

Repository:

JAVASCRIPT Source

JAVASCRIPT

```
class UserRepository {

  async findById(id) {
    return db.users.findById(id);
  }

}
```

Service:

JAVASCRIPT Source

JAVASCRIPT

```
class UserService {

  constructor(userRepository) {
    this.userRepository = userRepository;
  }

  async getUser(id) {
    return this.userRepository.findById(id);
  }

}
```

এতে database implementation change করা সহজ হয়।

37.47 Service Layer Pattern

Business logic controller-এ সব লিখে না রেখে Service layer-এ রাখা হয়। Bad:

✦ TIERED ENTERPRISE ARCHITECTURE FLOW

```
Controller
├── validation
├── business logic
├── payment
├── database
└── email
```

Better:

✦ TIERED ENTERPRISE ARCHITECTURE FLOW

```
Controller
↓
Service
↓
Repository
↓
Database
```

37.48 MVC Pattern

Web development-এ খুব পরিচিত architecture:

✦ TIERED ENTERPRISE ARCHITECTURE FLOW

```
MVC
├── Model
├── View
└── Controller
```

◆ Model

Data/business model

◆ View

UI

◆ Controller

Request/interaction handle করে

37.49 MVC Example

✦ TIERED ENTERPRISE ARCHITECTURE FLOW

```
Browser
  ↓
Controller
  ↓
Model
  ↓
Database
  ↓
Controller
  ↓
View
  ↓
Browser
```

Traditional server-rendered applications-এ এটি খুব common

37.50 Layered Architecture

Full Stack project-এ:

✦ TIERED ENTERPRISE ARCHITECTURE FLOW

```
Presentation Layer
  ↓
Controller Layer
  ↓
Service Layer
  ↓
Repository Layer
  ↓
Database
```

এভাবে responsibility ভাগ করা যায়।

37.51 Separation of Concerns

Design Pattern-এর বড় উদ্দেশ্যগুলোর একটি:

💡 একটি অংশের কাজ যেন অন্য অংশের কাজের সাথে অপ্রয়োজনীয়ভাবে মিশে না যায়।

উদাহরণ:

🌟 SOFTWARE ARCHITECTURE DIAGRAM

Authentication
Payment
Email
Database
UI

সব এক function-এ না রেখে আলাদা responsibility।

37.52 DRY

DRY = Don't Repeat Yourself খারাপ:

JAVASCRIPT Source

JAVASCRIPT

```
function adminLogin() {  
  // same validation  
}  
  
function userLogin() {  
  // same validation  
}
```

Better:

JAVASCRIPT Source

JAVASCRIPT

```
function validateLogin(data) {  
  // shared validation  
}
```

তারপর:

JAVASCRIPT Source

JAVASCRIPT

```
validateLogin(adminData);  
validateLogin(userData);
```

37.53 KISS

KISS = Keep It Simple অতিরিক্ত complex architecture সবসময় ভালো নয়। যদি problem:

JAVASCRIPT Source

JAVASCRIPT

```
function add(a, b) {  
  return a + b;  
}
```

হয়, তাহলে ৫টি design pattern দরকার নেই! 😊 Pattern complexity তৈরি করার জন্য নয়।

37.54 YAGNI

YAGNI = You Aren't Gonna Need It যে feature এখন দরকার নেই, শুধু ভবিষ্যতে লাগতে পারে ভেবে architecture over-engineer করো না।

🌟 SOFTWARE ARCHITECTURE DIAGRAM

Need today
↓
Build today

37.55 Composition Over Inheritance

Modern JavaScript/React development-এ খুব গুরুত্বপূর্ণ। Inheritance:

JAVASCRIPT Source

JAVASCRIPT

```
class Admin extends User {}
```

Composition:

JAVASCRIPT Source

JAVASCRIPT

```
const admin = {
  ...user,
  permissions: ["read", "write"]
};
```

অথবা behavior compose করা। React নিজেও composition-এর ওপর অনেক বেশি জোর দেয়।

Part 06 — Modern React & Full-Stack Application Patterns (37.56 – 37.66)

37.56 React-এ Design Patterns

React development-এ তুমি দেখতে পাবে:

✦ SOFTWARE ARCHITECTURE DIAGRAM

Component **Composition**
Container/Presentational
Custom Hooks
Provider Pattern
Compound Components
Render Props
Higher-**Order** Components
Observer-like state updates
Strategy-like behavior selection

37.57 Custom Hook as Reusable Pattern

ধরো:

JAVASCRIPT Source

JAVASCRIPT

```
function useFetch(url) {

  const [data, setData] = useState(null);

  // fetch logic

  return data;
}
```

Multiple components:

✦ SOFTWARE ARCHITECTURE DIAGRAM

```
const users = useFetch("/api/users");
```

এতে fetching logic reuse করা যায়।

37.58 Provider Pattern

React Context:

✦ SOFTWARE ARCHITECTURE DIAGRAM

```
<AuthProvider>
  <App />
</AuthProvider>
```

তাইপার:

✦ SOFTWARE ARCHITECTURE DIAGRAM

```
const { user } = useAuth();
```

একটি shared context provide করা হয়। এটি Provider Pattern বোঝার জন্য usefull

37.59 Compound Component Pattern

Example:

✦ SOFTWARE ARCHITECTURE DIAGRAM

```
<Select>
  <Select.Option />
  <Select.Option />
</Select>
```

Parent component এবং child components একসাথে coordinated behavior দেয়। এটি reusable UI component library-তে খুব usefull

37.60 Design Pattern নির্বাচন কীভাবে করবে?

প্রথমে pattern খুঁজবে না। প্রথমে problem বুঝবে।

✦ SOFTWARE ARCHITECTURE DIAGRAM

```
Problem
↓
What changes?
↓
What is coupled?
↓
What should be reusable?
↓
Which pattern fits?
```

37.61 Problem → Pattern Cheat Sheet

Problem	সম্ভাব্য Pattern
Object creation complex	Factory / Builder
One shared instance	Singleton
Incompatible API	Adapter
Add behavior dynamically	Decorator
Complex system simplify	Facade
Object access intercept	Proxy
State change notification	Observer
Event communication	Pub/Sub
Multiple algorithms	Strategy
Undo/redo/action queue	Command
State-based behavior	State
Request through handlers	Chain of Responsibility
External dependency control	Dependency Injection
Database abstraction	Repository

37.62 Pattern Overuse

এটা খুব গুরুত্বপূর্ণ। ধরো:

JAVASCRIPT Source

JAVASCRIPT

```
function add(a, b) {  
  return a + b;  
}
```

এখানে:

★ SOFTWARE ARCHITECTURE DIAGRAM

Factory
Singleton
Observer
Strategy
Facade

সব লাগানো absurd! 😬 Design Pattern:

★ SOFTWARE ARCHITECTURE DIAGRAM

Problem solve করার tool

Architecture decorate করার জন্য নয়।

37.63 Design Pattern-এর সুবিধা

★ SOFTWARE ARCHITECTURE DIAGRAM

- ✓ Reusability
- ✓ Maintainability
- ✓ Scalability
- ✓ Testability
- ✓ Loose Coupling
- ✓ Better Organization
- ✓ Team Communication

37.64 অসুবিধা

Pattern ভুলভাবে ব্যবহার করলে:

★ SOFTWARE ARCHITECTURE DIAGRAM

- ✗ **Over-engineering**
- ✗ More code
- ✗ More abstraction
- ✗ Harder debugging
- ✗ Unnecessary complexity

তাই:

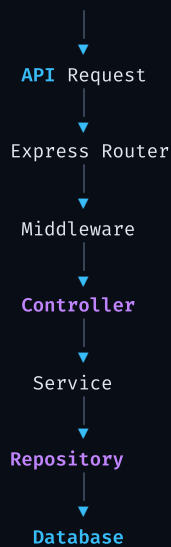
- 💡 Simple problem → Simple solution.

37.65 Real Full-Stack Architecture

তোমার Full Stack project-এর জন্য একটি practical architecture:

★ TIERED ENTERPRISE ARCHITECTURE FLOW

React Frontend



এখানে বিভিন্ন concept:

★ SOFTWARE ARCHITECTURE DIAGRAM

- Middleware
 - Chain of Responsibility
- Repository
 - Repository Pattern
- Service
 - Separation of Concerns
- Dependency Injection
 - Loose Coupling
- React Context
 - Provider Pattern
- API events
 - Observer / Pub-Sub concepts

37.66 Mini Real-World Example — E-Commerce Checkout Architecture

ধরো তুমি একটি E-commerce Order System বানাচ্ছে। Architecture:

✦ TIERED ENTERPRISE ARCHITECTURE FLOW

```
User
↓
OrderController
↓
OrderService
↓
PaymentStrategy
↓
PaymentGateway
↓
Repository
↓
Database
```

♦ Payment Strategy

JAVASCRIPT Source

JAVASCRIPT

```
const card = {
  pay(amount) {
    return `Card: ${amount}`;
  }
};

const bank = {
  pay(amount) {
    return `Bank: ${amount}`;
  }
};
```

♦ Service

JAVASCRIPT Source

JAVASCRIPT

```
class OrderService {

  constructor(paymentStrategy) {
    this.paymentStrategy = paymentStrategy;
  }

  checkout(amount) {
    return this.paymentStrategy.pay(amount);
  }

}
```

Use:

JAVASCRIPT Source

JAVASCRIPT

```
const service =
  new OrderService(card);

console.log(service.checkout(2000));
```

Output:

✦ SOFTWARE ARCHITECTURE DIAGRAM

Card: 2000

Payment method change:

JAVASCRIPT Source

JAVASCRIPT

```
const service =
  new OrderService(bank);
```

মূল OrderService code পরিবর্তন করতে হয়নি। এটাই Strategy + Dependency Injection-এর practical combination।

37.67

🔥 MUST KNOW — ডিজাইন প্যাটার্নের ২৫টি মূল ভিত্তি ও নীতি

সফটওয়্যার ইঞ্জিনিয়ারিং ও ফুলস্ট্যাক ডেভেলপমেন্টে সিনিয়র লেভেলে উত্তীর্ণ হতে এই ২৫টি কনসেপ্ট অত্যন্ত জরুরি:

★ 1. Creational & Structural

- **Factory:** অবজেক্ট তৈরির লজিক সেন্ট্রালাইজড করা
- **Singleton:** পুরো অ্যাপ্লিকেশনে শেয়ার্ড একটিমাত্র ইনস্ট্যান্স
- **Builder:** ধাপে ধাপে জটিল অবজেক্ট কনস্ট্রাকশন
- **Adapter:** ইনকম্প্যাটিবল ইন্টারফেসকে রূপান্তর
- **Decorator:** অবজেক্ট পরিবর্তন ছাড়া ফিচার যুক্ত
- **Facade:** জটিল সাবসিস্টেমের সহজ এন্ট্রি পয়েন্ট
- **Proxy:** অবজেক্ট অপারেশন ইন্টারসেপ্ট ও ক্যাশিং
- **Module:** গ্রাইডেট স্কোপ ও পাবলিক API এক্সপোজ

★ 2. Behavioral Patterns

- **Observer:** ডিপেন্ডেন্ট অবজেক্টে স্টেট চেঞ্জ ব্রডকাস্ট
- **Pub/Sub:** ইভেন্ট বাস দিয়ে সম্পূর্ণ ডিকাপল্ড কমিউনিকেশন
- **Strategy:** রানটাইমে অ্যালগরিদম বা পলিমেট পরিবর্তন
- **Command:** অ্যাকশনকে অবজেক্ট বানিয়ে কিউ বা আনডু
- **State:** ইন্টারনাল স্টেট অনুযায়ী অবজেক্টের আচরণ বদল
- **Chain of Resp.:** হ্যান্ডলার চেইন দিয়ে রিকোয়েস্ট পাস (Middleware)
- **Template Method:** স্কেলিটন ক্লাস দিয়ে সাবক্লাসে স্টেপ নির্ধারণ
- **Iterator:** সিকোয়েন্সের ভেতরের ডেটা ট্রাভার্সাল

★ 3. Architecture & Clean Code

- **Dependency Injection (DI):** বাইরে থেকে ডিপেন্ডেন্সি পুশ
- **Repository:** ডাটাবেস এক্সেস বিজনেস লজিক থেকে পৃথক
- **Service Layer:** কোর বিজনেস রুলসের সেন্ট্রাল লেয়ার
- **MVC:** Model, View এবং Controller সেপারেশন
- **Separation of Concerns:** প্রতিটি লেয়ারের নির্দিষ্ট কাজ
- **DRY, KISS, YAGNI:** সিম্পল ও আননেসেসারি কোড যুক্ত রাখা
- **Composition over Inheritance:** নমনীয় অবজেক্ট কম্পোজিশন
- **Pragmatism:** প্রয়োজনের অতিরিক্ত প্যাটার্ন ব্যবহার পরিহার

37.68

🧠 Quick Cheat Sheet — ডিজাইন প্যাটার্ন নির্বাচন নির্দেশিকা

Pattern Name	Category	Primary Intent	Production Real-World Example
Factory Pattern	Creational	অবজেক্ট তৈরির মেকানিজম এনক্যাপসুলেট করা	<code>document.createElement()</code> , Auth User Factory
Singleton Pattern	Creational	একটিমাত্র ইনস্ট্যান্স নিশ্চিত ও শেয়ার করা	Database Connection Pool, Global State Manager
Builder Pattern	Creational	ধাপে ধাপে জটিল কনফিগারেশন তৈরি	Knex/Prisma Query Builder, Form Builder
Adapter Pattern	Structural	ভিন্ন ইন্টারফেসকে প্রত্যাশিত ফরম্যাটে মেলানো	Legacy Payment API to Modern Interface
Decorator Pattern	Structural	অবজেক্ট মডিফাই না করে ডায়নামিক আচরণ বৃদ্ধি	Express Middleware, HOC (Higher-Order Components)
Facade Pattern	Structural	জটিল সাবসিস্টেমের সামনে একটি সহজ API দেওয়া	Axios facade over <code>fetch/XMLHttpRequest</code>
Proxy Pattern	Structural	অবজেক্টের অ্যাক্সেস নিয়ন্ত্রণ বা ক্যাশিং	Vue 3 Reactive System, Validation Proxy
Observer Pattern	Behavioral	১-টু-মেনি অবজেক্ট নোটিফিকেশন সিস্টেম	DOM <code>addEventListener</code> , MobX Observables
Pub/Sub Pattern	Behavioral	মধ্যবর্তী চ্যানেল দ্বারা সম্পূর্ণ ডিকাপল্ড ইভেন্ট	Node.js <code>EventEmitter</code> , Redux Actions, Redis Pub/Sub
Strategy Pattern	Behavioral	রানটাইমে অ্যালগরিদম অদলবদল করা	Multi-gateway Payment (Card/bKash/Bank)
Command Pattern	Behavioral	রিকোয়েস্টকে অবজেক্টে রূপান্তর (Undo/Redo)	Rich Text Editor actions, Task Queues
State Pattern	Behavioral	অবজেক্টের অভ্যন্তরীণ স্টেট অনুযায়ী আচরণ বদলানো	Order Status Machine (Pending -> Paid -> Shipped)
Chain of Responsibility	Behavioral	হ্যান্ডলার চেইনে রিকোয়েস্ট পাস করা	Express/Koa middleware pipeline
Dependency Injection	Architectural	ক্লাসের বাইরে থেকে ডিপেন্ডেন্সি পাস করা	NestJS DI Container, Unit Testing Mocking
Repository Pattern	Architectural	ডাটা অ্যাক্সেস লজিক বিজনেস লজিক থেকে পৃথক	ORM Repositories (Prisma, TypeORM)
Service Layer	Architectural	বিজনেস লজিক এক জায়গায় সেন্ট্রালাইজ করা	Enterprise Node.js / NestJS Backend

Lab 1: Factory Pattern — Dynamic User Creation with Role Permissions

সমস্যা ও লক্ষ্য: একটি `createUser("admin" | "user" | "guest")` Factory তৈরি করুন যা ইউজারের রোল অনুযায়ী নির্দিষ্ট পারমিশন (`canEdit`, `canDelete`, `canView`) যুক্ত অবজেক্ট রিটার্ন করবে।

PRODUCTION SOLUTION: LAB 1

JAVASCRIPT

```
function createUser(role, name) {
  const baseUser = { name, role, createdAt: new Date().toISOString() };

  switch (role) {
    case 'admin':
      return {
        ...baseUser,
        permissions: { canView: true, canEdit: true, canDelete: true },
        describe() { return `${this.name} [ADMIN] - Full Access`; }
      };
    case 'user':
      return {
        ...baseUser,
        permissions: { canView: true, canEdit: true, canDelete: false },
        describe() { return `${this.name} [USER] - Edit Access`; }
      };
    case 'guest':
    default:
      return {
        ...baseUser,
        permissions: { canView: true, canEdit: false, canDelete: false },
        describe() { return `${this.name} [GUEST] - Read-Only Access`; }
      };
  }
}

// Verification
const admin = createUser('admin', 'Tanvir');
const guest = createUser('guest', 'Rahim');
console.log(admin.describe()); // Tanvir [ADMIN] - Full Access
console.log('Guest Delete Allowed:', guest.permissions.canDelete); // false
```

💡 **আর্কিটেকচারাল ব্যাখ্যা:** Factory Pattern অবজেক্ট তৈরির কমপ্লেক্স লজিক ও রুলস একটি সাধারণ ফাংশনের ভেতরে এনক্যাপসুলেট করে দেয়, ফলে ক্লায়েন্ট কোডে বারবার `if/else` চেক করার দরকার হয় না।

Lab 2: Singleton Pattern — Centralized Application Logger

BEGINNER

সমস্যা ও লক্ষ্য: একটি Logger ক্লাস তৈরি করুন যেখানে যতবারই `new Logger()` করা হোক না কেন, সবসময় একই শেয়ার্ড ইনস্ট্যান্স রিটার্ন হবে (`logger1 === logger2` হবে `true`)।

PRODUCTION SOLUTION: LAB 2

JAVASCRIPT

```
class Logger {
  constructor() {
    if (Logger.instance) {
      return Logger.instance;
    }
    this.logs = [];
    Logger.instance = this;
  }

  log(message) {
    const timestamp = new Date().toLocaleTimeString();
    const entry = `[${timestamp}] ${message}`;
    this.logs.push(entry);
    console.log(entry);
  }

  getLogCount() {
    return this.logs.length;
  }
}

// Verification
const logger1 = new Logger();
const logger2 = new Logger();

logger1.log('Application started successfully');
logger2.log('Database connected');

console.log('Is Identical Instance?', logger1 === logger2); // true
console.log('Total Logs Recorded:', logger1.getLogCount()); // 2
```

💡 **আর্কিটেকচারাল ব্যাখ্যা:** Singleton Pattern নিশ্চিত করে যে কোনো রিসোর্স-ইনটেনসিভ বা স্টেটফুল ক্লাসের পুরো অ্যাপ্লিকেশনে শুধুমাত্র একটিমাত্র ইনস্ট্যান্স বিদ্যমান থাকবে।

Lab 3: Adapter Pattern — Bridging Legacy API to Modern Interface

BEGINNER

সমস্যা ও লক্ষ্য: একটি পুরোনো সিস্টেমের API মেথড `oldAPI.getUserData()` রিটার্ন করে `{ first_name, last_name, user_email }`। এটিকে ক্লায়েন্টের প্রত্যাশিত আধুনিক ইন্টারফেস `userService.getUser()` অর্থাৎ `{ fullName, email }` ফরম্যাটে রূপান্তর করতে একটি Adapter লিখুন।

PRODUCTION SOLUTION: LAB 3

JAVASCRIPT

```
// Legacy Service (Cannot modify directly)
class LegacyUserAPI {
  getUserData() {
    return {
      first_name: 'Shariar',
      last_name: 'Ahamed',
      user_email: 'shariar@example.com'
    };
  }
}

// Adapter implementing Modern Target Interface
class UserAdapter {
  constructor(legacyApi) {
    this.legacyApi = legacyApi;
  }

  getUser() {
    const data = this.legacyApi.getUserData();
    return {
      fullName: `${data.first_name} ${data.last_name}`,
      email: data.user_email
    };
  }
}

// Client Usage
const legacyAPI = new LegacyUserAPI();
const userService = new UserAdapter(legacyAPI);

const user = userService.getUser();
console.log('Modern Format:', user);
// Output: Modern Format: { fullName: 'Shariar Ahamed', email: 'shariar@example.com' }
```

💡 **আর্কিটেকচারাল ব্যাখ্যা:** Adapter Pattern বিদ্যমান কোনো ক্লাসের কোড পরিবর্তন না করেই দুটি অসঙ্গতিপূর্ণ ইন্টারফেসের মধ্যে একটি সুসংগত সেতু তৈরি করে দেয়।

Lab 4: Strategy Pattern — Interchangeable Multi-Gateway Payment System

INTERMEDIATE

সমস্যা ও লক্ষ্য: একটি ই-কমার্স পেমেন্ট সিস্টেম তৈরি করুন যাতে Card, Bank Transfer এবং Mobile Banking (bKash/Nagad) স্ট্র্যাটেজি আলাদা অবজেক্ট হিসেবে থাকবে এবং পেমেন্ট সার্ভিস রানটাইমে কৌশল অদলবদল করতে পারবে।

PRODUCTION SOLUTION: LAB 4

JAVASCRIPT

```
// 1. Concrete Strategies
const CardPaymentStrategy = {
  pay(amount) {
    return `Paid ₳${amount} via Credit/Debit Card (Gateway Fee: 1.5%)`;
  }
};

const BankTransferStrategy = {
  pay(amount) {
    return `Paid ₳${amount} via Direct Bank Wire (Routing: 092834)`;
  }
};

const MobileBankingStrategy = {
  pay(amount) {
    return `Paid ₳${amount} via bKash/Nagad Wallet (Instant SMS TrxID)`;
  }
};

// 2. Context Service
class CheckoutService {
  constructor(strategy) {
    this.strategy = strategy;
  }

  setStrategy(newStrategy) {
    this.strategy = newStrategy;
  }

  processPayment(amount) {
    return this.strategy.pay(amount);
  }
}

// Usage
const checkout = new CheckoutService(CardPaymentStrategy);
console.log(checkout.processPayment(5000));

// Switch strategy at runtime without breaking CheckoutService
checkout.setStrategy(MobileBankingStrategy);
console.log(checkout.processPayment(1200));
```

💡 **আর্কিটেকচারাল ব্যাখ্যা:** Strategy Pattern অ্যালগরিদম বা বিজনেস মেকানিজমকে আলাদা ক্লাসে বিভক্ত করে, যার ফলে মূল কনটেক্সট ক্লাস পরিবর্তন না করেই নতুন নতুন পেমেন্ট মেথড যুক্ত করা যায় (Open/Closed Principle)।

Lab 5: Observer Pattern — Real-Time Notification Broadcast

INTERMEDIATE

সমস্যা ও লক্ষ্য: এমন একটি Subject তৈরি করুন যা মেসেজ আসলে তার সকল রেজিস্টার্ড Observer (User 1, User 2, User 3)-কে স্বয়ংক্রিয়ভাবে নোটিফাই করবে।

PRODUCTION SOLUTION: LAB 5

JAVASCRIPT

```
class NotificationSubject {
  constructor() {
    this.observers = [];
  }

  subscribe(observer) {
    this.observers.push(observer);
  }

  unsubscribe(observer) {
    this.observers = this.observers.filter(obs => obs !== observer);
  }

  notify(message) {
    this.observers.forEach(observer => observer.update(message));
  }
}

class UserObserver {
  constructor(username) {
    this.username = username;
  }

  update(message) {
    console.log(`[Notification to ${this.username}]: ${message}`);
  }
}

// Execution
const channel = new NotificationSubject();
const user1 = new UserObserver('Abrar');
const user2 = new UserObserver('Fatima');

channel.subscribe(user1);
channel.subscribe(user2);

channel.notify('🔥 JavaScript Master Chapter 37 is now available!');
// Both Abrar and Fatima receive notifications simultaneously
```

💡 **আর্কিটেকচারাল ব্যাখ্যা:** Observer Pattern-এ সাবজেক্টের স্টেটে কোনো পরিবর্তন ঘটলে সকল সাবস্ক্রাইবার অবজেক্ট তা তৎক্ষণাৎ জানতে পারে, যা রিয়েল-টাইম ডাটা সিস্টেম/নোটিফিকেশনে অপরিহার্য।

Lab 6: Pub/Sub Pattern — Fully Decoupled Global Event Bus

INTERMEDIATE

সমস্যা ও লক্ষ্য: একটি গ্লোবাল EventBus তৈরি করুন যাতে subscribe(event, callback) এবং publish(event, data) মেথড থাকবে, যেখানে পাবলিশার এবং সাবস্ক্রাইবার একে অপরকে সরাসরি চেনে না।

PRODUCTION SOLUTION: LAB 6

JAVASCRIPT

```
class EventBus {
  constructor() {
    this.events = {};
  }

  subscribe(event, callback) {
    if (!this.events[event]) {
      this.events[event] = [];
    }
    this.events[event].push(callback);

    // Return unsubscribe function
    return () => {
      this.events[event] = this.events[event].filter(cb => cb !== callback);
    };
  }

  publish(event, data) {
    if (this.events[event]) {
      this.events[event].forEach(callback => callback(data));
    }
  }
}

// Global Bus Usage
const bus = new EventBus();

const unsubAuth = bus.subscribe('user:login', user => {
  console.log(`Auth System: Welcome back ${user.name}!`);
});

bus.subscribe('user:login', user => {
  console.log(`Analytics System: Logged in event for ID ${user.id}`);
});

bus.publish('user:login', { id: 101, name: 'Tanvir' });
unsubAuth(); // Cleanly detach auth listener
```

💡 **আর্কিটেকচারাল ব্যাখ্যা:** Pub/Sub-এ একটি সেন্ট্রাল ব্রোকার (Event Bus) থাকে। ফলে সেন্ডার ও রিসিভার কেউ কাউকে চেনে না; তারা কেবল ইভেন্টের নামের ভিত্তিতে স্বাধীনভাবে যোগাযোগ করে।

Lab 7: Dependency Injection — Decoupling Service from Concrete Database

ADVANCED

সমস্যা ও লক্ষ্য: এমন একটি UserService তৈরি করুন যা কনস্ট্রাক্টরের মাধ্যমে ডাটাবেস ইন্সট্যান্স ইনজেক্ট করে নেয়। এর মাধ্যমে প্রোডাকশন DB এবং টেস্ট মক DB উভয়েই যাতে একই সার্ভিস ব্যবহার করতে পারে তা দেখান।

PRODUCTION SOLUTION: LAB 7

JAVASCRIPT

```
// Database Implementations
class MongoDBDatabase {
  find(id) {
    return { id, name: 'Prod User', source: 'MongoDB Atlas' };
  }
}

class MockTestDatabase {
  find(id) {
    return { id, name: 'Test Fixture User', source: 'In-Memory Mock' };
  }
}

// Service with Injected Dependency
class UserService {
  constructor(database) {
    this.db = database; // Injected externally
  }

  getUserProfile(id) {
    const record = this.db.find(id);
    return `User: ${record.name} (Retrieved via ${record.source})`;
  }
}

// In Production
const prodService = new UserService(new MongoDBDatabase());
console.log(prodService.getUserProfile(1));

// In Unit Testing
const testService = new UserService(new MockTestDatabase());
console.log(testService.getUserProfile(1));
```

💡 **আর্কিটেকচারাল ব্যাখ্যা:** Dependency Injection (DI) ক্লাসের ভেতরে সরাসরি new Class() কল বন্ধ করে বাইরে থেকে ইনস্ট্যান্স সাপ্লাই করে। এটি কোডকে 100% টেস্টেবল ও মডুলার বানায়।

Lab 8: Repository Pattern — Clean Architecture Data Access Layer

ADVANCED

সমস্যা ও লক্ষ্য: Controller -> Service -> Repository -> Database এই লেয়ারড আর্কিটেকচার মেনে একটি সম্পূর্ণ ইউজার রেজিস্ট্রেশন ও ফেচিং সিস্টেম লিখুন।

PRODUCTION SOLUTION: LAB 8

JAVASCRIPT

```
// 1. Repository Layer (Only talks to DB)
class UserRepository {
  constructor() {
    this.storage = new Map();
  }

  create(user) {
    this.storage.set(user.id, user);
    return user;
  }

  findById(id) {
    return this.storage.get(id) || null;
  }
}

// 2. Service Layer (Pure Business Logic & Validations)
class UserService {
  constructor(userRepo) {
    this.userRepo = userRepo;
  }

  register(id, name, email) {
    if (!email.includes('@')) throw new Error('Invalid email address');
    return this.userRepo.create({ id, name, email });
  }

  getDetails(id) {
    const user = this.userRepo.findById(id);
    if (!user) throw new Error('User not found');
    return user;
  }
}

// 3. Controller Layer (HTTP / Client Interface)
class UserController {
  constructor(userService) {
    this.userService = userService;
  }

  handleRegister(id, name, email) {
    try {
      const created = this.userService.register(id, name, email);
      return { status: 201, data: created };
    } catch (err) {
      return { status: 400, error: err.message };
    }
  }
}

// Wire-up
const repo = new UserRepository();
const service = new UserService(repo);
const controller = new UserController(service);

console.log(controller.handleRegister(1, 'Kabir', 'kabin@code.bd'));
```

💡 **আর্কিটেকচারাল ব্যাখ্যা:** Repository Pattern বিজনেস লজিকে ডাটা স্টোরেজ মেকানিজম থেকে সম্পূর্ণ আলাদা রাখে। কাল যদি SQL থেকে NoSQL-এ শিফট করা হয়, তবে শুধুমাত্র Repository ক্লাস পরিবর্তন করলেই চলে।

Lab 9: Full Enterprise Mini-Project — E-Commerce Checkout System

ADVANCED

সমস্যা ও লক্ষ্য: Factory, Strategy, Service, Repository এবং Observer প্যাটার্নগুলোর সমন্বয়ে একটি সম্পূর্ণ এন্ড-টু-এন্ড E-Commerce Checkout System তৈরি করুন।

PRODUCTION SOLUTION: LAB 9

JAVASCRIPT

```
// 1. Strategy: Payment Methods
const BkashStrategy = { pay: amount => `Bkash: ট${amount} confirmed` };
const CardStrategy = { pay: amount => `Visa/Mastercard: ট${amount} settled` };

// 2. Observer: Order Notification Event Bus
class OrderNotifier {
  constructor() { this.listeners = []; }
  onOrderSuccess(fn) { this.listeners.push(fn); }
  broadcast(order) { this.listeners.forEach(fn => fn(order)); }
}

// 3. Repository: Order Storage
class OrderRepository {
  constructor() { this.orders = []; }
  save(order) { this.orders.push(order); return order; }
}

// 4. Service: Business Logic & Orchestration
class OrderService {
  constructor(orderRepo, notifier) {
    this.orderRepo = orderRepo;
    this.notifier = notifier;
  }

  checkout(orderId, items, amount, paymentStrategy) {
    // 1. Process payment via strategy
    const paymentReceipt = paymentStrategy.pay(amount);

    // 2. Persist order via repository
    const order = this.orderRepo.save({
      id: orderId,
      items,
      amount,
      paymentReceipt,
      status: 'PAID',
      date: new Date().toISOString()
    });

    // 3. Notify all listeners via observer
    this.notifier.broadcast(order);
    return order;
  }
}

// --- Application Assembly & Execution ---
const notifier = new OrderNotifier();
notifier.onOrderSuccess(order => console.log(`[SMS Gateway]: Order #${order.id} paid ট${order.amount}`));
notifier.onOrderSuccess(order => console.log(`[Warehouse]: Pack items for Order #${order.id}`));

const repo = new OrderRepository();
const orderService = new OrderService(repo, notifier);

const completedOrder = orderService.checkout(
  'ORD-9821',
  ['JavaScript Master Book', 'Coffee Mug'],
  1450,
  BkashStrategy
);

console.log('Final Order Result:', completedOrder.status);
```

💡 **আর্কিটেকচারাল ব্যাখ্যা:** বাস্তব সফটওয়্যারে কোনো একক প্যাটার্ন একা চলে না। বিভিন্ন প্যাটার্নের সমন্বয়ে একটি আর্কিটেকচার তৈরি হয় যা অত্যন্ত স্কেলেবল, পরিবর্তনশীল এবং এন্টারপ্রাইজ-রেডি।

পুরো Chapter 37-এর ডিজাইন প্যাটার্ন ও সফটওয়্যার আর্কিটেকচার একনজরে:

★ DESIGN PATTERNS TAXONOMY & ARCHITECTURE MAP

DESIGN PATTERNS

CREATIONAL

Factory
Builder
Singleton
Prototype

STRUCTURAL

Adapter
Decorator
Facade
Proxy

BEHAVIORAL

Observer
Pub/Sub
Strategy
Command
State
Chain

ARCHITECTURE

Controller Service Repository

Dependency
Injection

Database/API

🏆 এক লাইনে Chapter 37: Design Pattern কোনো মুখস্থ কোড নয়—এটি একটি প্রমাণিত আর্কিটেকচারাল সমাধান যা কোডকে সহজে পরিবর্তনযোগ্য, বাগ-মুক্ত এবং প্রোডাকশন স্কেলে বজায় রাখতে সাহায্য করে।

★ ROADMAP CONTINUITY PIPELINE

Chapter 36: Modern ECMAScript / ES2020+ Features



Optional Chaining, Nullish Coalescing, Top-level Await, toSorted, Object.groupBy



Chapter 37: JavaScript Design Patterns (THIS CHAPTER)



Creational, Structural, Behavioral, MVC, Dependency Injection & Repository



Chapter 38: Advanced JavaScript for React



Reactivity Engine, Custom Hooks Architecture, State Machines & Virtual DOM Deep Dive